

ZAK MCP Gateway

How it works, in plain words: every technique, a picture of it, an everyday comparison, and the measured results.

88%

fewer tokens for tool definitions on every model request (GitHub, 26 tools)

95%

fewer tokens in tool results across 7 real GitHub calls

93%

fewer input tokens over a whole 7-step agent task

94%

still saved when the model uses prompt caching (cost-weighted)

Measured on 4 Oct 2026 with the real GitHub MCP server against the public repo modelcontextprotocol/servers, counted with the o200k tokenizer. Details and limits are in section 5.

1. The problem in one minute

An AI agent uses tools (GitHub, files, databases) through **MCP**. The model is billed per **token**, which is roughly a short piece of a word. MCP wastes tokens in two places:

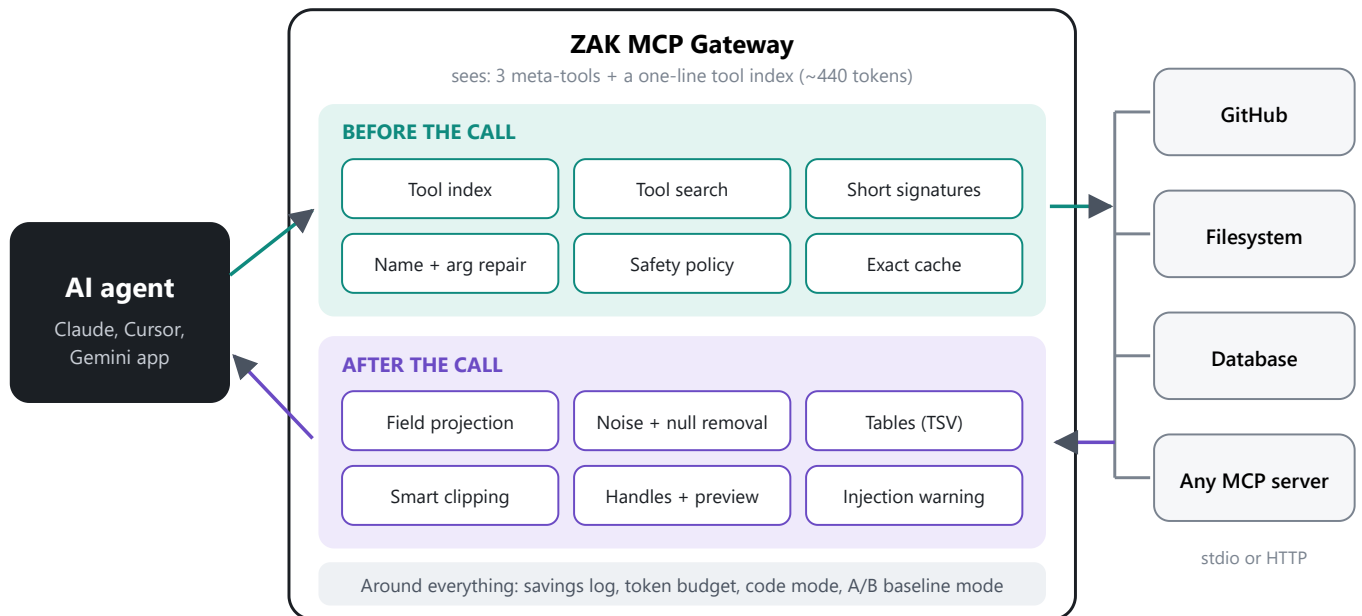
Before the call: the manuals. Every tool's full JSON description is sent with *every* request, even if the agent uses only one or two tools. GitHub alone is 3,600 tokens; GitHub plus the filesystem server is about 7,500.

After the call: the results. Asking for 10 commits returns about 18,000 tokens of JSON: avatars, API links, IDs, pretty-printing. The agent needed about 800 tokens of it, and the rest stays in the conversation for every later turn.

Analogy: You ask a hotel front desk for "a taxi". Before you can speak, they hand you the manual for every service in the hotel. When the taxi arrives, they also give you the driver's full employment file. The gateway is a good concierge: a one-page directory, a short answer, and the file stays in the back office if you ever need it.

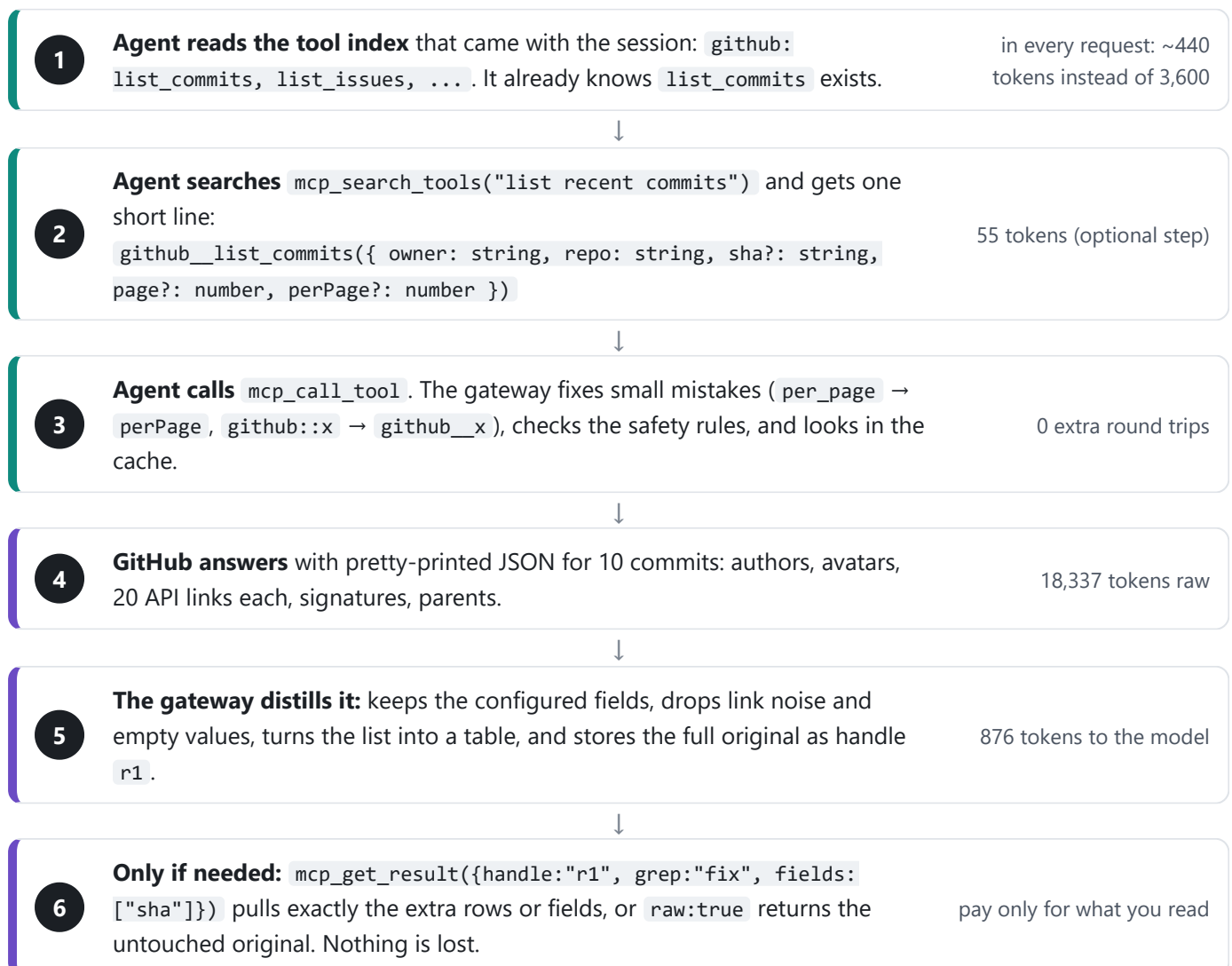
2. The big picture

The gateway is one MCP server that the agent connects to. Behind it sit all your real MCP servers. Teal techniques (and the teal request arrows) shrink what goes **into** the model before a call; purple techniques shrink what comes **back** after it.



3. One request, step by step

Real example from the benchmark: *"show the last 10 commits of modelcontextprotocol/servers"*.



What the model actually receives in step 5 (columns shortened here for print):

```
sha      commit.message      commit.author.name  commit.author.date  html_url
5abed86... Merge pull request #4939 from ...policy-to-main  Cliff Hall          2026-10-04T04:01:54Z  https://github.com/...
a207787... chore: sync the bug report form with v2/main... cliffhall           ...                  https://github.com/...
... 8 more rows
[r1: default fields, noise keys removed; full data via mcp_get_result]
```

4. Every technique, with an everyday comparison

Technique	Where	What it does (simply)	Real-world analogy
Meta-tools	before	The agent sees 3 tools (search, call, get_result) instead of every server's tools.	One concierge desk instead of 40 service counters.
Tool index	before	One line per server listing tool names, sent as server instructions. The model knows what exists without the manuals, and often skips the search.	The building directory in the lobby.
Tool search (BM25)	before	Finds the right tool from plain words, with synonyms like "PR" → pull request and "ticket" → issue. Right tool in the top 3 for 32 of 32 test queries.	Asking a librarian instead of reading every catalog card.
Short signatures	before	A long JSON schema becomes one TypeScript-style line. Parameter hints and limits are kept. Schemas too complex to shorten safely keep the full (minified) version.	A recipe card instead of the cookbook chapter.
Name + argument repair	before	Accepts <code>github::x</code> or <code>x</code> ; renames <code>per_page</code> → <code>perPage</code> ; turns <code>"5"</code> into <code>5</code> ; on a real mistake it returns the signature, so the next try is right.	A postal clerk who fixes a misspelled street instead of returning the letter.
Exact cache	before	The same read with the same arguments within 5 minutes is answered from memory. Never used for writes, and any write clears that server's cache.	A cafe that remembers your usual order, and forgets it when the menu changes.
Safety policy	before	Read-only mode, per-server read-only, allow and deny lists, and writes need <code>confirm: true</code> after the user agrees. Tools are classed read or write from MCP annotations, then verbs; unknown counts as write.	A bank teller who asks for your signature before a withdrawal.
Field projection	after	Keeps only the fields asked for (<code>project_fields</code>), or per-tool defaults from the config. Works through lists: <code>commit.author.name</code> on every commit.	A highlighter on a long report.
Noise + null removal	after	Drops API link fields (<code>*_url</code>), internal IDs, and empty or null values.	Throwing away junk mail before handing over the post.
Tables (TSV)	after	A list of records is sent as a table, so key names appear once, not on every row. Used only when it is smaller.	A spreadsheet instead of one letter per customer.
Smart clipping	after	Only when a result is too big: long text in lists is cut short; a single document keeps as much as fits.	Reading the first chapters, with a bookmark for the rest.
Handles + preview	after	A big result stays in the gateway. The model gets the first rows plus a handle like <code>r3</code> , then pages, greps, or picks fields.	A coat-check ticket: your coat is safe; collect it only if you need it.

Injection warning	after	Results containing text like "ignore previous instructions" are flagged as untrusted data.	An "unverified sender" stamp on a note from a stranger.
Savings log + report	around	Every call is logged (raw vs sent tokens) to a JSONL file; <code>npm run report</code> recomputes totals. Live view: resource <code>zak://stats</code> .	An electricity meter: you see what you saved.
Token budget	around	Optional session limit: at 80% results get half the room; at 100% the gateway asks the model to wrap up.	A prepaid phone plan that warns you before it runs out.
Code mode (opt-in)	around	The model writes a short script that calls several read tools inside the gateway; only the final answer returns. Not a security sandbox, so it is off by default.	Sending a runner to visit five shops and bring back one receipt.
Tool preload (agent UI)	around	The first user message is searched up front and the best 3 signatures go into the system prompt, saving one model round trip.	A waiter who brings water before you ask.
Passthrough baseline	around	<code>--passthrough</code> runs the same servers as plain MCP, for honest before/after tests on the same scale.	Weighing yourself before and after on the same scale.

Why fewer round trips matter: each time the model calls a tool, the *whole* conversation is sent again. A 18,000-token result read once is paid for on every later turn. Keeping it out of the conversation saves it many times over. That is why the agent-loop saving (93%) is larger than any single step suggests.

5. Measured results (live GitHub MCP)

Same downstream server, same calls, two modes: **Direct MCP** (`--passthrough`) and **Gateway**. Command: `npm run bench:github`.

Direct MCP Gateway each panel uses its own scale

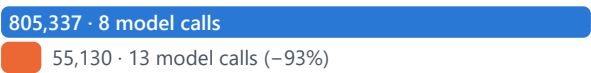
Tool definitions per request



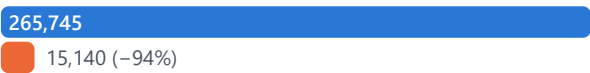
7 tool results (+ search for gateway)



Whole 7-step agent task: input tokens



Same task with prompt caching (cost-weighted)



Agent task model: every request re-sends tool definitions plus all earlier results; the gateway adds one search request per new tool.
Caching model: previously sent prefix billed at 10%, new tokens at 125% (Anthropic-style cache write/read prices).

Call	Direct MCP	Gateway	Search used	What the gateway did
list_commits (10)	18,337	876	55	default fields → table
list_commits (30), agent asked for sha + message	61,589	1,280	0	projection → table
list_issues (20 open)	32,013	890	267	default fields → table
list_pull_requests (10 closed)	20,929	827	229	default fields → table
search_repositories (10)	3,470	485	257	wrapper → meta line + table
get_file_contents (README, 8.7 KB)	2,498	1,212	198	kept most of the text, handle for the rest
list_commits again (same args)	18,337	876	0	served from cache (no GitHub call)
Total	157,173	6,446	1,006	

With GitHub and the filesystem server together (40 tools), tool definitions drop from about 7,500 to about 560 tokens per request (gateway's own estimate, `npm run test:github`).

Read these numbers honestly

- **Small setups can cost more.** With 5 small tools the gateway's ~410 definition tokens exceed the ~280 of direct MCP. The gateway pays off with many tools or big results.
- **Extra round trips.** Searching adds model requests (13 vs 8 here). They are cheap because the context is small, but they add latency. The tool index and tool preload let the model skip most searches.
- **Tokenizer.** Counts use the o200k tokenizer as a stand-in; Claude and Gemini count a little differently, but ratios hold. The LLM-in-the-loop check (real Gemini usage numbers, both modes) is in `agent-ui` and needs your Gemini API key.
- **Search accuracy** (32/32 in the top 3) was measured on a query set used while tuning synonyms, so treat it as a regression check, not independent proof.

- **Task success** is not measured by token counts. Default projections hide fields; the handle keeps them reachable, but the agent must ask.

6. Bugs found and fixed in this upgrade

Problem	Effect	Fix
Cache key used <code>JSON.stringify(obj, keysArray)</code>	Every call hashed to the same key, so one cached result could be returned for a different tool	Stable, sorted serialization of tool + args; regression test
<code>isIdempotent = readOnly false</code>	Caching silently off for almost every server; name check never ran	Read/write classifier: MCP annotations, then verbs, unknown = write
Results wrapped in <code>JSON.stringify(result, null, 2)</code>	Double-encoded JSON (<code>\</code> " everywhere) plus indentation, bigger than the raw result	Return the distilled content directly
<code>"<YOUR_GITHUB_TOKEN>"</code> placeholder in config	Overrode the real environment token → GitHub 401	Placeholders ignored, <code>\${VAR}</code> secrets, warning for hard-coded tokens
Tests and live script used <code>github::</code> ; code used <code>github__</code>	"Tool not found"	One separator (<code>__</code> , valid for every LLM) plus tolerant name lookup
Live test sent <code>per_page</code> , the tool wants <code>perPage</code>	Silently ignored: 30 commits instead of 3	Argument repair renames near-miss names and reports it
Projection through arrays	<code>commit.message</code> on a list returned nothing	Path-tree projection that walks lists
Agent UI "tokens saved" = <code>35000 + 0.1 × input</code>	Made-up number	Real A/B: same prompt through gateway and passthrough, Gemini's own usage numbers
Agent UI counted only the last model call; handled one function call per turn; sent raw <code>\$schema</code> to Gemini <code>parameters</code>	Undercounted tokens; dropped parallel calls; schema errors	Sum all calls; answer all calls; use <code>parametersJsonSchema</code>
Unused config: <code>cache.enabled</code> , <code>maxEntries</code> , <code>minScore</code> ; unused TSV converter	Settings did nothing	All wired in (LRU cache, relative score floor, TSV renderer)

7. Plan coverage (Architecture & Build Plan)

Phase	Status	Notes
1. Core gateway	done	Config, parallel downstream connections (stdio and HTTP), catalog, search, signatures, 3 meta-tools. Client sees ~440 tokens with GitHub (target $\approx$ 400).
2. Result handling	done	Token estimate, null removal, TSV, handles with paging/grep/fields/raw, projection. A 61k-token result returns 1.3k.
3. Stats	done	JSONL log (not SQLite: simpler, no dependency, recomputable), CLI report, session budget, <code>zak://stats</code> .
4. Safety	done	Read-only, confirm writes, allow/deny, injection warning, secrets from env.
5. Benchmark	partial	Live GitHub A/B done. Still open: 10–20 tasks over 3+ servers with a real model and success rates (the agent UI A/B mode is ready for it).
6. Advanced	partial	Done: exact cache, code mode (opt-in), tool preload, configured default fields. Not done: cheap-model router (belongs in the host app, not the gateway) and learned field pruning.

Tests use Node's built-in test runner instead of vitest, so no new dependency was added. The only new package is `js-tiktoken`, as a dev dependency for the benchmark.

8. Try it yourself

```
npm install && npm test          # 27 tests: unit + full MCP round trip with a mock server
npm run test:github              # live: CLI over stdio → GitHub MCP (no token needed for public repos)
npm run bench:github             # live A/B token benchmark (writes .zak-gateway/benchmark-github.json)
npm run report                   # savings from the log
cd agent-ui && npm run dev        # Gemini agent, "Both (A/B)" mode, real usage numbers
```